

PEARLS AQI PREDICTOR

End-to-End Air Quality Index Forecasting System for Peshawar

PROJECT REPORT

Prepared By:
Iqra Jan

September 6th, 2026

Table of Contents

1. EXECUTIVE SUMMARY	7
2. PROBLEM STATEMENT.....	8
3. SYSTEM ARCHITECTURE OVERVIEW.....	8
4. DATA COLLECTION	9
DATA SOURCE	10
5. WEATHER DATA.....	10
6. AIR-QUALITY DATA	10
7. HISTORICAL DATA BACKFILL	10
8. DATA PROCESSING.....	11
EXPLORATORY DATA ANALYSIS.....	11
9. FEATURE ENGINEERING	12
10. MODEL FEATURES	12
WEATHER FEATURES	12
POLLUTANT FEATURES	12
AQI LAG FEATURES	13
11. POLLUTANT LAG FEATURES.....	13
12. ROLLING FEATURES.....	13
13. CHANGE FEATURES.....	13
14. TIME-BASED FEATURES	13
15. TARGET VARIABLE.....	13
16. DATA LEAKAGE PREVENTION	14
17. FEAST FEATURE STORE.....	14
18. POSTGRESQL AND SUPABASE	14
19. MODEL DEVELOPMENT	14
TRAINING MODELS	15
REGISTER THE SELECTED MODELS.....	15
20. TRAINING, VALIDATION, AND TESTING	15
21. MODEL COMPARISON	16
22. CHAMPION MODEL	16
23. MLFLOW MODEL REGISTRY	16
24. FORECASTING METHODOLOGY	17

RECURSIVE FORECASTING.....	17
DAILY FORECAST.....	17
25. 72-HOUR EVALUATION.....	17
PERSISTENCE BASELINE	18
26. FASTAPI BACKEND.....	18
API ENDPOINTS	18
27. STREAMLIT DASHBOARD	18
DASHBOARD FEATURES.....	19
28. DAILY SHAP AGGREGATION.....	19
29. CI/CD ARCHITECTURE	19
FEATURE PIPELINE SCHEDULING LIMITATION.....	20
SCHEDULED TRAINING PIPELINE EXECUTION DELAY	20
30. FEATURE PIPELINE	20
START THE SCHEDULED WORKFLOW	21
SET UP THE EXECUTION ENVIRONMENT.....	21
LOAD CONFIGURATION AND SECRETS	21
COLLECT CURRENT ENVIRONMENTAL DATA	21
UPDATE THE HISTORICAL/CURRENT DATASET.....	21
VALIDATE THE RAW DATA	21
PERFORM FEATURE ENGINEERING	21
CREATE THE NEXT-HOUR TARGET WHERE REQUIRED	21
REMOVE INVALID ROWS CREATED BY TRANSFORMATIONS.....	22
VALIDATE THE PROCESSED FEATURE DATASET	22
PREPARE THE FEAST FEATURE DATASET	22
MATERIALIZE FEATURES INTO FEAST	22
VERIFY FEATURE AVAILABILITY	22
PUBLISH THE FEATURE ARTIFACT.....	22
31. TRAINING PIPELINE.....	22
START THE DAILY TRAINING WORKFLOW	22
SET UP THE TRAINING ENVIRONMENT	23
LOAD REQUIRED SECRETS AND CONFIGURATION	23
CONFIGURE CLOUD FEAST	23

CONFIGURE MLFLOW AND DAGSHUB.....	23
VERIFY THE MLFLOW TRACKING LOCATION	23
RETRIEVE THE LATEST SUCCESSFUL FEATURE ARTIFACT	23
VALIDATE THE TRAINING DATASET	23
SEPARATE MODEL FEATURES AND TARGET.....	23
PERFORM THE CHRONOLOGICAL DATA SPLIT	23
TRAIN MODELS	24
CALCULATE EVALUATION METRICS.....	24
COMPARE THE PRODUCTION CANDIDATES	24
LOG EXPERIMENTS TO MLFLOW	24
REGISTER THE PRODUCTION MODELS.....	24
SELECT THE CHAMPION MODEL	24
ASSIGN THE CHAMPION ALIAS	24
GENERATE THE FORECAST	24
VERIFY THE FORECAST OUTPUT	24
UPLOAD RELEVANT ARTIFACTS.....	24
VERIFY THE DEPLOYED PREDICTION SERVICE	25
32. PROJECT STRUCTURE	25
33. SEPARATION BETWEEN DEVELOPMENT AND PRODUCTION.....	26
34. VALIDATION AND TESTING	26
DATA VALIDATION.....	26
FEATURE VALIDATION	26
MODEL VALIDATION.....	26
APPLICATION VALIDATION.....	26
35. LIMITATIONS.....	27
LOCATION LIMITATION	27
FORECAST UNCERTAINTY.....	27
RECURSIVE ERROR	27
EXTERNAL DATA DEPENDENCY	27
DISTRIBUTION SHIFT.....	27
36. FUTURE IMPROVEMENTS	27
PREDICTION INTERVALS.....	27

MODEL DRIFT MONITORING	27
MULTI-CITY FORECASTING	27
AUTOMATED RETRAINING	28
BETTER MONITORING	28
37. COMPLETE TECHNOLOGY STACK	28
38. FINAL CONCLUSION	28

PROJECT INFORMATION

Project Title: Pearls AQI Predictor

Project Description: Peshawar Air Quality Monitoring and 3-Day AQI Forecasting System

Location: Peshawar, Pakistan

Forecast Horizon: 72 hours / 3 days

Primary Model: XGBoost Regression

Feature Store: Feast with PostgreSQL database

Model Registry: MLflow with DagsHub

Backend: FastAPI

Dashboard: Streamlit

Automation: GitHub Actions

Deployed Application:

[HTTPS://AQI-PREDICTOR-AXAEFUYOJZTZTPJGAZHDCQ.STREAMLIT.APP/](https://aqi-predictor-axaefuyojztztpjgazhdcq.streamlit.app/)

Source Code Repository:

[HTTPS://GITHUB.COM/IQRAJAN2024/AQI_PREDICTOR](https://github.com/IQRAJAN2024/AQI_PREDICTOR)

1. EXECUTIVE SUMMARY

Pearls AQI Predictor is an end-to-end machine learning system developed to monitor air quality in Peshawar and forecast the Air Quality Index (AQI) for the following three days. The system covers automated data collection, historical data backfilling, exploratory data analysis, data processing, feature engineering, feature-store management, model training, model evaluation, model registration, recursive forecasting, API serving, dashboard visualization, explainability, and CI/CD automation.

The system uses Open-Meteo as the primary source for historical and forecast weather and air-quality information. Approximately two years of hourly historical data were collected, covering the period from August 1, 2024 to August 1, 2026. The final merged historical dataset contains 17,544 hourly records and 20 columns. The raw data was validated for duplicate timestamps, missing values, chronological consistency, and expected schema before being used for subsequent analysis.

Exploratory Data Analysis (EDA) was performed before feature engineering to understand the structure and behavior of the collected data. The analysis examined pollutant distributions, AQI behavior, weather variables, temporal patterns, correlations, trends, and potential anomalies. This stage helped identify the characteristics of AQI variation and guided the selection of meaningful lag, rolling, change, weather, pollutant, and temporal features.

Following data processing and feature engineering, the processed dataset contains 17,471 usable rows and 79 columns. Exactly 70 features are used as model inputs. These features represent current environmental conditions together with historical AQI and pollutant information, rolling statistics, rate-of-change information, and temporal characteristics.

Four machine learning approaches were trained and evaluated during model development: Ridge Regression, Random Forest Regression, XGBoost Regression, and a Bidirectional Long Short-Term Memory (BiLSTM) model. The BiLSTM model was investigated as a sequence-based deep learning approach because AQI is a time-dependent forecasting problem. However, the final production model registry contains the three conventional regression models, namely Ridge Regression, Random Forest Regression, and XGBoost Regression. XGBoost produced the strongest documented test performance and was therefore selected as the champion production model.

The prediction target is the AQI for the following hour:

```
target_aqi = us_aqi.shift(-1)
```

The final XGBoost model achieved a test MAE of 1.3132, RMSE of 2.4022, and R^2 of 0.9943. It was registered through MLflow under the model name Pearls_AQI_XGBoost and assigned the production alias champion.

The production forecasting system generates 72 hourly predictions recursively. The latest observed AQI is used to initialize the forecast. After each prediction, the predicted AQI becomes

part of the state used for the next prediction. This approach prevents the production system from using future actual AQI values that would not be available at prediction time.

The prediction service is exposed through FastAPI, while Streamlit provides the user-facing dashboard. The dashboard displays the current AQI, recent AQI trends, three-day forecasts, model information, evaluation metrics, and SHAP-based explanations.

GitHub Actions automates the data and machine learning lifecycle. The feature pipeline is configured to run hourly, while the model training pipeline is configured for daily execution at 12:00 AM Pakistan Standard Time. The system also supports manual workflow execution when required.

2. PROBLEM STATEMENT

Air pollution is highly dynamic and changes according to pollutant concentrations, weather conditions, seasonal patterns, and recent air-quality conditions. A current AQI measurement describes the present state of air quality but does not provide information about what air quality may look like several hours or days into the future.

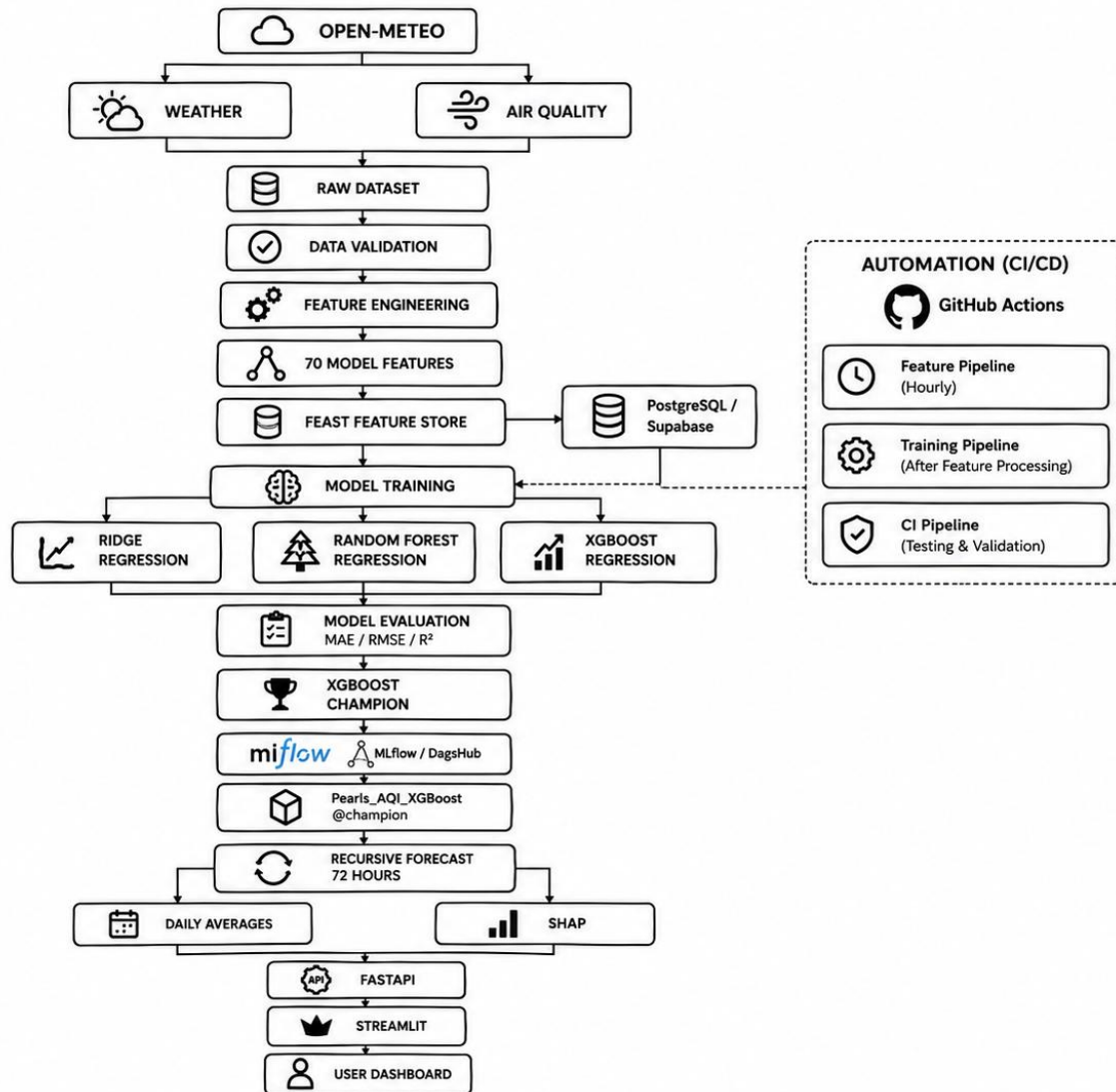
The purpose of this project is therefore to build a machine learning system capable of learning historical relationships between AQI, pollutants, weather conditions, and temporal patterns and using those relationships to predict future AQI. The system focuses specifically on Peshawar and produces a three-day forecast so that users can understand expected air-quality conditions in advance.

3. SYSTEM ARCHITECTURE OVERVIEW

The complete system follows an end-to-end architecture in which data is collected from Open-Meteo, validated and processed, analyzed through exploratory data analysis, transformed through feature engineering, and stored through Feast. The engineered features are used for model training and evaluation, while MLflow and DagsHub provide experiment tracking and model registry functionality.

The production prediction workflow retrieves the required features from the feature store and loads the registered champion model from MLflow. The prediction service is exposed through FastAPI, and the Streamlit dashboard communicates with the backend to display current conditions, forecasts, evaluation information, and explanations.

The overall flow can be summarized as:



4. DATA COLLECTION

The first stage of the system is the collection of historical and current environmental data. Because AQI depends on both atmospheric conditions and pollutant concentrations, the project collects weather and air-quality variables at hourly frequency.

The collection process was designed to provide a consistent timestamp-based dataset that could subsequently be used for exploratory analysis, feature engineering, and supervised machine learning.

DATA SOURCE

The project uses Open-Meteo as its primary data source. Open-Meteo provides both weather information and air-quality information, allowing the system to obtain the required environmental variables without depending on multiple unrelated data providers.

The project focuses on Peshawar, Pakistan, using the approximate coordinates:

Latitude: 34.008° N

Longitude: 71.5785° E

The same geographic location is used consistently throughout the historical data collection and forecasting process.

5. WEATHER DATA

Weather variables are included because atmospheric conditions influence pollutant concentration, dispersion, accumulation, and overall air-quality behavior. The weather dataset contains hourly environmental measurements that can provide useful information for predicting future AQI.

The main weather variables include temperature, relative humidity, mean sea-level pressure, precipitation, wind speed, and wind direction.

6. AIR-QUALITY DATA

The air-quality dataset contains the main pollutant measurements required for AQI forecasting. The pollutant variables include PM2.5, PM10, carbon monoxide, nitrogen dioxide, sulphur dioxide, and ozone.

In addition to pollutant concentrations, the dataset contains the overall US AQI and pollutant-specific AQI values. These include AQI values associated with PM2.5, PM10, carbon monoxide, nitrogen dioxide, sulphur dioxide, and ozone.

The availability of these variables provides both direct pollutant information and AQI-related context for feature engineering and model evaluation.

7. HISTORICAL DATA BACKFILL

A major requirement of the project was obtaining enough historical data for reliable time-series model training. The final historical backfill covered approximately two years, from August 1, 2024 to August 1, 2026, at hourly frequency.

The final merged raw dataset contained 17,544 rows and 20 columns. Validation showed zero duplicate timestamps and no missing values in the validated merged dataset.

The historical backfill was important because it provided sufficient information for learning daily patterns, seasonal behavior, AQI persistence, pollutant behavior, weather relationships, short-term changes, and longer-term rolling behavior.

8. DATA PROCESSING

After collection, the weather and air-quality datasets were merged using their timestamps. Maintaining a consistent hourly timestamp is important because the prediction target represents the AQI one hour into the future.

The processing pipeline performs several validation steps before the data is passed to feature engineering. These checks include timestamp validation, duplicate timestamp detection, missing-value detection, chronological ordering, expected column validation, and general data consistency checks.

The processed dataset is maintained at hourly frequency because the primary prediction task is next-hour AQI forecasting.

EXPLORATORY DATA ANALYSIS

Exploratory Data Analysis was performed after the initial data collection and validation but before feature engineering. The purpose of EDA was to understand the underlying structure of the AQI dataset and identify patterns that could guide the subsequent feature-engineering process.

The analysis examined the distributions of AQI and pollutant variables, including PM2.5, PM10, carbon monoxide, nitrogen dioxide, sulphur dioxide, and ozone. Descriptive statistics were used to understand the central tendency, spread, minimum and maximum values, and potential extreme observations within the dataset.

Temporal analysis was also performed because AQI is a time-dependent variable. Hourly, daily, weekly, monthly, and seasonal patterns were examined to determine whether AQI and pollutant levels changed systematically over time. These observations supported the later inclusion of hour, day, month, day-of-week, and weekend-related features.

Correlation analysis was used to investigate relationships between AQI, pollutants, and weather variables. This helped determine which environmental variables could provide useful predictive information. The analysis also provided a basis for creating pollutant lag variables, AQI lag variables, rolling averages, and change features.

The EDA stage also helped identify the importance of recent AQI history. AQI often demonstrates temporal persistence, meaning that the current AQI is strongly related to the AQI observed during recent hours. This observation motivated the use of multiple AQI lag variables and rolling statistics.

Overall, EDA provided an understanding of the dataset before transformation and ensured that feature engineering was guided by observed data characteristics rather than being performed without prior analysis.

9. FEATURE ENGINEERING

Feature engineering is one of the most important components of the project. Raw measurements alone do not provide sufficient historical context for a multi-hour forecasting system. Therefore, the project transforms the validated hourly dataset into a richer feature representation.

Historical lag features, rolling features, change features, pollutant history, weather variables, and temporal features were created to provide the models with information about both the current environmental state and recent historical behavior.

After feature engineering and removal of rows that could not be used because of lag, rolling-window, and target-shifting requirements, the processed dataset contains 17,471 usable rows and 79 columns. Exactly 70 columns are used as model inputs.

10. MODEL FEATURES

The 70 model features are grouped into several categories. These categories represent current environmental conditions, recent historical conditions, pollutant behavior, AQI persistence, temporal behavior, and changes in environmental measurements.

WEATHER FEATURES

The weather feature group contains the following variables:

- temperature_2m
- relative_humidity_2m
- pressure_msl
- precipitation
- wind_speed_10m
- wind_direction_10m

POLLUTANT FEATURES

The pollutant feature group contains:

- PM2.5
- PM10
- Carbon monoxide
- Nitrogen dioxide
- Sulphur dioxide
- Ozone

AQI LAG FEATURES

Historical AQI values are used to capture temporal persistence. The project includes AQI lag variables representing AQI values from 1, 3, 6, 12, 24, 48, and 72 hours earlier.

These features allow the model to learn how recent AQI affects the next-hour AQI and provide historical context during forecasting.

11. POLLUTANT LAG FEATURES

Historical pollutant measurements are also included through lag features. Lag variables are created for PM2.5, PM10, carbon monoxide, nitrogen dioxide, sulphur dioxide, and ozone.

The purpose of these variables is to capture the relationship between recent pollutant concentrations and future AQI. Pollutant lag information can help the model distinguish between a temporarily high pollutant concentration and a persistent pollution condition.

12. ROLLING FEATURES

Rolling statistics are used to represent recent pollution conditions over multiple time windows. AQI rolling means include 3-hour, 6-hour, 12-hour, and 24-hour averages.

PM2.5 and PM10 rolling means include 3-hour, 6-hour, and 24-hour averages. The system also calculates 24-hour rolling means for carbon monoxide, nitrogen dioxide, sulphur dioxide, and ozone.

Rolling features reduce the influence of isolated hourly fluctuations and provide the models with information about the recent average pollution state.

13. CHANGE FEATURES

The project also uses AQI and pollutant change features to represent how rapidly environmental conditions are changing.

AQI change features include changes over 1 hour, 3 hours, and 6 hours. Similar change information is calculated for pollutant concentrations.

These features help the model distinguish between a stable AQI condition and a rapidly increasing or decreasing pollution condition.

14. TIME-BASED FEATURES

AQI patterns can vary depending on the time of day, day of the week, month, and seasonal conditions. The project therefore includes temporal features such as hour, day, month, day of week, and weekend-related information.

These variables allow the models to learn recurring temporal patterns in AQI behavior.

15. TARGET VARIABLE

The target variable represents the AQI for the following hour. It is created using:

```
target_aqi = us_aqi.shift(-1)
```

Therefore, if the current observation is at time T, the target corresponds to AQI at time T+1 hour.

This transformation converts the historical time-series dataset into a supervised learning problem in which the model receives information available at time T and predicts the AQI at time T+1.

16. DATA LEAKAGE PREVENTION

Data leakage is a major risk in time-series machine learning because information from the future must not be used to predict the past or current state.

The project uses several mechanisms to prevent leakage. First, the data is divided chronologically rather than randomly. Second, rolling features are calculated using historical information rather than future observations. Third, the target variable is excluded from the model input features.

The production forecasting system provides exactly 70 model features to the model. The latest observed AQI is maintained separately as the recursive prediction state. During multi-hour forecasting, future actual AQI values are not used.

17. FEAST FEATURE STORE

Feast is used as the feature-store layer of the system. The main feature view is `aqi_features`, and the entity used by the feature store is `location_id`.

The feature store provides a consistent mechanism for accessing the features required by both training and prediction. The project contains a feature view, a training feature service, prediction context, and online feature serving.

The training feature service is: `aqi_model_features`

The prediction context is: `aqi_prediction_context`

The feature-store TTL is configured for approximately seven days.

18. POSTGRESQL AND SUPABASE

PostgreSQL is used as the cloud database infrastructure for the Feast configuration. Supabase provides the managed PostgreSQL environment, allowing the feature-store infrastructure to persist independently from the local development environment.

The project also maintains a local Feast configuration for development and testing. This creates a separation between the local environment used for experimentation and testing and the cloud environment used for production-oriented feature serving.

19. MODEL DEVELOPMENT

The project investigated multiple machine learning approaches to determine which modeling technique was most suitable for AQI prediction.

Four models were trained during the model-development stage:

- Ridge Regression
- Random Forest Regression
- XGBoost Regression
- Bidirectional Long Short-Term Memory (BiLSTM)

Ridge Regression was included as a regularized linear baseline. Random Forest Regression was used as a tree-based ensemble approach capable of modeling nonlinear relationships. XGBoost Regression was included as a gradient-boosting approach that is effective for structured tabular data.

BiLSTM was also trained because AQI is a temporal forecasting problem and recurrent neural networks are capable of learning patterns from sequences. The BiLSTM approach was therefore investigated as a deep learning alternative that could capture temporal dependencies in the historical AQI and environmental data.

Although all four approaches were trained and evaluated during development, only the three conventional regression models, Ridge Regression, Random Forest Regression, and XGBoost Regression were included in the final MLflow model-registration workflow. XGBoost achieved the strongest documented test performance among the production candidates and was therefore selected as the champion model.

TRAINING MODELS

The model-training stage used the engineered dataset containing 17,471 usable observations. The training process was designed as a chronological machine learning workflow so that future observations were not used during training.

REGISTER THE SELECTED MODELS

Ridge Regression, Random Forest Regression, and XGBoost Regression are included in the final MLflow model-registration workflow. The BiLSTM remains part of the model-development experiment but is not included in the final production registry.

20. TRAINING, VALIDATION, AND TESTING

The processed dataset contains 17,471 usable rows. The data was split chronologically into training, validation, and testing subsets rather than being randomly shuffled.

Chronological Dataset Split:

Training: 12,229 rows — 70%

Validation: 2,621 rows — 15%

Testing: 2,621 rows — 15%

Total: 17,471 rows — 100%

Chronological splitting is particularly important for AQI forecasting because randomly mixing past and future observations can produce unrealistic evaluation results. The chronological split ensures that the models are evaluated on later observations that occur after the training period.

The validation set is used during model development, while the test set provides the final documented performance comparison.

21. MODEL COMPARISON

The final documented test results for the three production regression models are:

Ridge Regression:

MAE = 2.7992

RMSE = 4.3062

$R^2 = 0.9816$

Random Forest Regression:

MAE = 2.6022

RMSE = 4.4868

$R^2 = 0.9800$

XGBoost Regression:

MAE = 1.3132

RMSE = 2.4022

$R^2 = 0.9943$

XGBoost achieved the lowest MAE and RMSE and the highest R^2 among the three registered regression models. Its MAE of 1.3132 indicates a relatively small average absolute prediction error, while its RMSE of 2.4022 indicates that the model also performed strongly when larger errors were considered. The R^2 value of 0.9943 indicates that the model explains a very high proportion of the variation in the test target.

22. CHAMPION MODEL

The production model selected for the system is: Pearls_AQI_XGBoost

The production alias is: champion

The model can be referenced through: models:/Pearls_AQI_XGBoost@champion

The use of the champion alias means that the prediction service does not need to be manually updated every time a new validated version is selected. Instead, the service loads the model currently assigned to the production alias.

23. MLFLOW MODEL REGISTRY

MLflow is used for experiment tracking, model logging, model versioning, model registry, production model selection, and evaluation metric storage.

DagsHub provides the cloud MLflow tracking and registry layer. During the training pipeline, the trained models and their evaluation information are logged and the selected production model is registered.

The final production XGBoost model is registered under the name Pearls_AQI_XGBoost and identified through the champion alias.

24. FORECASTING METHODOLOGY

The production system generates 72 hourly AQI predictions, representing three days of 24-hourly predictions.

The forecasting pipeline uses:

`HISTORY_HOURS = 96`

`FORECAST_HOURS = 72`

The 96-hour historical context provides sufficient recent information to calculate the required lag and rolling features before generating the future predictions.

RECURSIVE FORECASTING

The forecast is generated recursively because future actual AQI values are unavailable at prediction time.

The process begins by obtaining the latest observed AQI. The forecasting system then constructs the first future feature vector and sends the 70 model features to the XGBoost champion model. After generating the first future AQI prediction, that prediction is stored and used as the AQI state for the next forecasting step.

The lag and rolling information are then updated using the newly generated prediction, and the next feature vector is constructed. This process continues recursively until all 72 future hourly AQI values have been generated.

This approach provides a realistic production forecasting procedure because it does not depend on future observed AQI values.

DAILY FORECAST

The 72 hourly predictions are aggregated into three daily averages. The dashboard therefore presents the forecast in a simplified three-day format consisting of Day 1, Day 2, and Day 3 average AQI.

The daily aggregation makes the forecast easier for users to interpret while retaining the underlying hourly predictions in the backend.

25. 72-HOUR EVALUATION

The project includes a dedicated 72-hour evaluation pipeline to assess the performance of the forecasting system across the complete forecast horizon. The evaluation records predicted AQI,

actual AQI, and persistence AQI. It calculates RMSE, MAE, and R^2 . Performance can also be evaluated separately for Day 1, Day 2, and Day 3.

PERSISTENCE BASELINE

The evaluation pipeline also includes a persistence forecast as a baseline. Persistence assumes that the most recent observed AQI remains unchanged and uses that value as the prediction for future observations.

This is a simple but useful baseline because it provides a reference point against which the machine learning model can be compared. A useful forecasting model should ideally provide better predictive performance than simply assuming that AQI will remain unchanged.

26. FASTAPI BACKEND

FastAPI is used as the backend API layer of the production system. The backend provides access to current AQI information, recent AQI trends, future forecasts, model information, SHAP explanations, and health information.

The API creates a separation between the machine learning infrastructure and the user interface. This allows the Streamlit dashboard to communicate with the prediction service without directly interacting with the underlying model and feature-store infrastructure.

API ENDPOINTS

The main API endpoints are:

GET /health

This endpoint is used to check whether the backend service is operating correctly.

GET /current

This endpoint returns the current AQI and related information.

GET /current/trend

This endpoint returns recent AQI trend information.

GET /forecast

This endpoint returns the three-day forecast together with model information.

GET /shap/{forecast_day}

This endpoint returns SHAP explanations for the selected forecast day.

27. STREAMLIT DASHBOARD

Streamlit is used to create the user-facing dashboard. The dashboard does not bypass the production backend architecture. Instead, the communication flow is:

Streamlit → FastAPI → Prediction and ML Infrastructure

This architecture keeps the frontend separate from the backend and allows the prediction service to remain independently accessible.

The dashboard is designed specifically for Peshawar AQI monitoring and three-day forecasting.

DASHBOARD FEATURES

The dashboard provides a range of information designed to make the prediction system understandable to end users. It includes:

- Current AQI
- AQI category
- Current timestamp
- Recent AQI trend
- Three-day AQI forecast
- Daily forecast cards
- Model information
- Model metrics
- Forecast evaluation metrics
- SHAP explanations
- Refresh functionality

Timestamps are converted into Pakistan Standard Time for user-friendly presentation.

28. DAILY SHAP AGGREGATION

The forecasting system produces hourly predictions, and SHAP explanations can therefore also be generated for individual forecast hours.

Displaying separate explanations for all 72 forecast hours would create a large amount of information for users. To make the explanations easier to understand, the dashboard aggregates approximately 24 hourly explanations for each forecast day.

This produces a daily explanation that summarizes the most influential features contributing to the forecast instead of displaying 24 separate explanations for every day.

29. CI/CD ARCHITECTURE

GitHub Actions automates the machine learning lifecycle of the project. The repository contains separate workflows for the hourly feature pipeline and the daily training pipeline.

The feature workflow is responsible for updating the data, validating it, performing feature engineering, validating the resulting feature dataset, and materializing the required features into the cloud feature store.

The training workflow is responsible for obtaining the latest successful feature data, preparing the feature-store environment, training the models, evaluating them, registering the validated production model, generating the forecast, and verifying the deployed prediction service.

FEATURE PIPELINE SCHEDULING LIMITATION

The feature pipeline is configured in GitHub Actions to execute automatically on an hourly schedule. However, scheduled GitHub Actions workflows are not guaranteed to start at the exact scheduled time. Due to GitHub's server-side scheduling behavior and runner availability, scheduled execution may be delayed or vary from the configured time.

Therefore, although the system is designed for hourly automated feature updates, the actual execution time can vary. The pipeline is scheduled hourly rather than being guaranteed to start at exactly the configured minute. This limitation is related to the GitHub Actions infrastructure rather than the workflow logic itself.

The feature pipeline also supports manual execution, providing an alternative when an immediate feature update is required.

SCHEDULED TRAINING PIPELINE EXECUTION DELAY

The proposed system uses GitHub Actions to automate the daily AQI prediction pipeline. The workflow is scheduled to start at **12:00 AM PKT** each day. However, scheduled execution on a shared cloud-based CI/CD platform is not guaranteed to begin at the exact configured time. Runner availability, platform workload, scheduling delays, and other GitHub Actions service-side factors may cause the workflow to start later than scheduled.

Furthermore, the complete pipeline requires approximately **34 minutes** to complete because it performs several sequential operations, including feature data retrieval and validation, Feast feature-store materialization, model training, historical 72-hour evaluation, MLflow metric logging, forecast generation, artifact management, and API deployment and verification.

Consequently, there may be a period during which the dashboard continues to display the previous successful forecast rather than a newly generated forecast. This means that the system cannot guarantee that an updated forecast will be available exactly at 12:00 AM every day.

This limitation primarily concerns **automation timeliness and cloud infrastructure availability**, rather than the predictive capability of the trained model. The system remains operational using the most recent successfully generated forecast until the next pipeline execution completes.

30. FEATURE PIPELINE

The feature pipeline is responsible for keeping the environmental data and engineered features up to date. It is designed to execute automatically every hour through GitHub Actions.

The configured GitHub Actions cron expression is: 15 * * * *

This schedules the workflow for approximately minute 15 of every UTC hour. Since Pakistan Standard Time is UTC+5, the corresponding local scheduled time is approximately minute 15 of each hour after applying the five-hour offset.

The feature pipeline performs the following steps:

START THE SCHEDULED WORKFLOW

GitHub Actions starts the feature workflow according to the configured hourly cron schedule. The workflow can also be started manually when an immediate update is required.

SET UP THE EXECUTION ENVIRONMENT

The workflow checks out the project repository, installs the required Python version and project dependencies, and prepares the environment required for data processing and Feast operations.

LOAD CONFIGURATION AND SECRETS

Required configuration values and secrets are loaded securely from the GitHub Actions environment. These values allow the workflow to communicate with the required external services and cloud infrastructure without placing credentials directly in the source code.

COLLECT CURRENT ENVIRONMENTAL DATA

The pipeline communicates with Open-Meteo to obtain the latest available weather and air-quality observations for Peshawar. The data is collected at hourly frequency and contains the environmental variables required by the feature-engineering process.

UPDATE THE HISTORICAL/CURRENT DATASET

The newly collected observations are combined with the existing environmental data used by the project. Timestamp-based processing ensures that the records remain aligned to the hourly forecasting frequency.

VALIDATE THE RAW DATA

Before feature engineering, the pipeline validates the collected data. The validation checks include the presence of required columns, timestamp validity, duplicate timestamps, missing values, chronological ordering, and general dataset consistency.

PERFORM FEATURE ENGINEERING

The validated data is transformed into the feature representation used by the forecasting system. Weather variables, pollutant variables, AQI lag features, pollutant lag features, rolling statistics, change features, and time-based features are generated.

CREATE THE NEXT-HOUR TARGET WHERE REQUIRED

For training data, the target is created using the next-hour AQI transformation:

```
target_aqi = us_aqi.shift(-1)
```

This allows the resulting feature dataset to be used for supervised next-hour AQI prediction.

REMOVE INVALID ROWS CREATED BY TRANSFORMATIONS

Lag features, rolling windows, and the shifted target require historical or future neighboring observations. Rows that cannot provide the required information are removed so that the resulting feature dataset contains valid training observations.

VALIDATE THE PROCESSED FEATURE DATASET

The pipeline checks that the processed dataset contains the expected feature columns and that the required model inputs are available. It verifies that the model feature set contains exactly 70 features and that the resulting data does not contain unexpected missing values.

PREPARE THE FEAST FEATURE DATASET

The validated features are prepared according to the Feast feature-store configuration. The `aqi_features` feature view uses `location_id` as the entity for the environmental observations.

MATERIALIZE FEATURES INTO FEAST

The current feature information is materialized into the cloud Feast feature store. This makes the latest feature values available to downstream training and prediction components.

VERIFY FEATURE AVAILABILITY

The workflow performs checks to confirm that the required feature-store configuration and feature data are available after materialization.

PUBLISH THE FEATURE ARTIFACT

The resulting processed feature dataset or artifact is made available for the downstream daily training workflow. This allows the training workflow to retrieve the latest successful feature output without depending on the exact execution time of the previous feature workflow.

31. TRAINING PIPELINE

The training pipeline is responsible for periodically retraining and evaluating the machine learning models using the latest successful feature data. Unlike the feature pipeline, which executes hourly, the training pipeline executes once per day.

The training pipeline is designed to execute at 12:00 AM Pakistan Standard Time. Because GitHub Actions cron schedules use UTC, midnight in Pakistan Standard Time corresponds to 19:00 UTC on the previous calendar day.

The configured schedule is therefore: `0 19 * * *`

The training pipeline performs the following steps:

START THE DAILY TRAINING WORKFLOW

GitHub Actions starts the training workflow according to the daily cron schedule. The workflow can also be manually triggered for testing, maintenance, or immediate retraining.

SET UP THE TRAINING ENVIRONMENT

The workflow checks out the latest project code, installs the required Python dependencies, and prepares the environment required for model training, Feast access, evaluation, and MLflow operations.

LOAD REQUIRED SECRETS AND CONFIGURATION

Required credentials and configuration values for cloud services, Feast, MLflow, DagsHub, and deployment verification are loaded securely from GitHub Actions secrets and environment variables.

CONFIGURE CLOUD FEAST

The workflow prepares the cloud Feast configuration and verifies that the feature-store dependencies and configuration are available.

CONFIGURE MLFLOW AND DAGSHUB

MLflow credentials and tracking configuration are loaded so that model experiments, metrics, artifacts, versions, and registry information can be stored in the cloud tracking environment provided through DagsHub.

VERIFY THE MLFLOW TRACKING LOCATION

Before training begins, the workflow verifies that MLflow is configured to use the expected cloud tracking location rather than accidentally logging the production training run only to a local environment.

RETRIEVE THE LATEST SUCCESSFUL FEATURE ARTIFACT

The training workflow retrieves the latest successful output produced by the feature pipeline. This is important because the feature workflow is scheduled hourly and GitHub Actions may execute scheduled workflows later than their configured time.

The training workflow therefore operates independently of the exact timing of an individual hourly feature run and uses the latest successful feature artifact available for training.

VALIDATE THE TRAINING DATASET

The retrieved feature dataset is checked before training. The pipeline verifies that the dataset exists, the required target is available, the 70 model features are present, the data is appropriately ordered, and invalid or missing training values are not passed to the models.

SEPARATE MODEL FEATURES AND TARGET

The 70 engineered model features are separated from the next-hour AQI target represented by `target_aqi`. The target is not included among the model inputs.

PERFORM THE CHRONOLOGICAL DATA SPLIT

The 17,471 usable observations are divided into training, validation, and test sets using chronological ordering. The split contains 12,229 training rows, 2,621 validation rows, and 2,621 test rows.

TRAIN MODELS

The models are trained and their predictions are evaluated against the validation and test data using the defined performance metrics.

CALCULATE EVALUATION METRICS

Predictions from the trained models are evaluated using MAE, RMSE, and R^2 . These metrics provide information about average absolute error, sensitivity to larger errors, and the proportion of target variation explained by the model.

COMPARE THE PRODUCTION CANDIDATES

Ridge Regression, Random Forest Regression, and XGBoost Regression are compared as the production candidates.

LOG EXPERIMENTS TO MLFLOW

Training information, evaluation metrics, model artifacts, and relevant run information are logged to MLflow. This provides experiment tracking and allows model versions to be identified and managed.

REGISTER THE PRODUCTION MODELS

The selected conventional regression models—Ridge Regression, Random Forest Regression, and XGBoost Regression—are registered through the MLflow model registry.

SELECT THE CHAMPION MODEL

XGBoost is selected as the champion model because it provides the strongest documented performance among the registered production candidates.

ASSIGN THE CHAMPION ALIAS

The selected XGBoost version is assigned the champion alias under the model name `Pearls_AQI_XGBoost`. The production prediction service can therefore load the model using the alias rather than depending on a manually hard-coded version number.

GENERATE THE FORECAST

After successful model registration, the workflow generates the three-day forecast using the current production model. The forecasting system uses 96 hours of historical context and recursively generates 72 future hourly AQI predictions.

VERIFY THE FORECAST OUTPUT

The workflow checks that the forecast is generated successfully and contains the expected 72-hour prediction horizon and three-day aggregation.

UPLOAD RELEVANT ARTIFACTS

Relevant training outputs, evaluation results, and generated artifacts can be uploaded by the workflow for inspection and later reference.

VERIFY THE DEPLOYED PREDICTION SERVICE

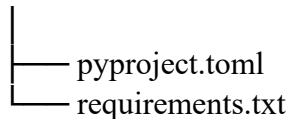
The workflow performs deployment-level checks against the FastAPI prediction service to verify that the backend is operating and that the deployed system can retrieve the expected prediction information.

32. PROJECT STRUCTURE

The repository is organized into separate directories for automation, feature-store configuration, models, notebooks, source code, scripts, and testing.

AQI_Predictor/

```
├── .github/
│   ├── workflows/
│   ├── ci.yml
│   ├── feature_pipeline.yml
│   └── training_pipeline.yml
├── feature_repo/
│   ├── feature_repo/
│   ├── feature_store.cloud.yaml
│   ├── feature_store.local.yaml
│   ├── features.py
│   └── test_feast.py
├── models/
├── notebooks/
│   ├── eda.ipynb
│   ├── model_training.ipynb
│   └── three_day_forecasting.ipynb
├── scripts/
│   └── configure_feast.py
├── src/
│   ├── api/
│   ├── dashboard/
│   ├── data/
│   ├── evaluation/
│   ├── features/
│   ├── models/
│   └── prediction/
└── tests/
```



33. SEPARATION BETWEEN DEVELOPMENT AND PRODUCTION

A major design principle of the project is the separation between experimentation and production.

Jupyter notebooks are used for exploration, visualization, exploratory data analysis, experimentation, model development, and comparison. These notebooks provide a flexible environment for investigating the dataset and testing different approaches.

Production Python modules are responsible for automated data collection, feature engineering, model training, evaluation, prediction, and API serving.

The project also maintains separate local and cloud configurations for Feast, allowing development and testing to be performed without directly modifying the production-oriented environment.

34. VALIDATION AND TESTING

The system contains validation at multiple stages of the machine learning lifecycle. This layered validation approach helps detect problems before they propagate into downstream components.

DATA VALIDATION

Data validation includes checks for missing values, duplicate timestamps, timestamp validity, chronological ordering, and expected schema.

FEATURE VALIDATION

Feature validation checks that exactly 70 model features are available, required columns exist, feature values are not missing, the target is available when required for training, and the dataset remains chronologically ordered.

MODEL VALIDATION

Model validation includes train, validation, and test evaluation using MAE, RMSE, and R^2 . The models are compared to determine which approach provides the strongest predictive performance.

APPLICATION VALIDATION

Application validation includes verification of the FastAPI health endpoint, API error handling, dashboard API connectivity, forecast retrieval, and SHAP retrieval.

The CI/CD workflows also perform verification steps after important stages, including feature-store checks, model registry checks, forecast checks, and deployment checks.

35. LIMITATIONS

Although the system performs strongly on the documented test dataset, several limitations remain.

LOCATION LIMITATION

The current system is primarily designed and validated for Peshawar. It has not yet been fully validated as a multi-city Pakistani AQI forecasting model.

FORECAST UNCERTAINTY

The current production forecast provides point predictions. It does not yet provide formal prediction intervals or uncertainty estimates. Therefore, users cannot directly see the expected uncertainty around each predicted AQI value.

RECURSIVE ERROR

Because predictions are recursively fed into later forecasting steps, prediction errors can accumulate over the 72-hour horizon. Consequently, longer-horizon predictions may be less accurate than short-horizon predictions.

EXTERNAL DATA DEPENDENCY

The system depends on Open-Meteo for environmental and forecast information. If the upstream data is unavailable, delayed, or inaccurate, the quality and freshness of the AQI forecast can also be affected.

DISTRIBUTION SHIFT

Future air-quality behavior may differ from the historical training distribution. Changes in pollution sources, weather patterns, unusual environmental events, or extreme pollution episodes may therefore produce larger prediction errors.

36. FUTURE IMPROVEMENTS

Several improvements can be introduced in future versions of the system.

PREDICTION INTERVALS

Future versions can provide uncertainty ranges around AQI forecasts so that users can understand not only the predicted AQI but also the confidence associated with the prediction.

MODEL DRIFT MONITORING

The system can automatically monitor feature distribution changes, AQI distribution changes, prediction errors, and model performance degradation.

MULTI-CITY FORECASTING

The system can be extended to cities such as Islamabad, Lahore, Karachi, Quetta, Rawalpindi, and Multan. The existing `location_id` entity in Feast provides a foundation for this extension.

AUTOMATED RETRAINING

Future retraining can be triggered based on model performance degradation, data drift, feature drift, or the availability of new historical data rather than relying only on a fixed schedule.

BETTER MONITORING

A dedicated monitoring layer could track data freshness, API availability, forecast freshness, model version, prediction latency, and forecast accuracy.

37. COMPLETE TECHNOLOGY STACK

<i>Category</i>	<i>Technology / Tool</i>
<i>Programming Language</i>	Python
<i>Data Source</i>	Open-Meteo
<i>Data Processing</i>	pandas, NumPy
<i>Machine Learning</i>	scikit-learn
<i>Deep Learning</i>	TensorFlow / Keras
<i>Feature Store</i>	Feast
<i>Database</i>	PostgreSQL
<i>Managed Database</i>	Supabase
<i>Experiment Tracking</i>	MLflow
<i>Model Registry</i>	MLflow
<i>Cloud ML Platform</i>	DagsHub
<i>Backend</i>	FastAPI
<i>ASGI Server</i>	Uvicorn
<i>Dashboard</i>	Streamlit
<i>Visualization</i>	Plotly
<i>Explainability</i>	SHAP
<i>CI/CD</i>	GitHub Actions
<i>Testing</i>	PyTest
<i>Version Control</i>	Git / GitHub
<i>Development Environment</i>	VS Code

38. FINAL CONCLUSION

Pearls AQI Predictor is a complete machine learning application designed for AQI monitoring and three-day forecasting in Peshawar. The project integrates data engineering, exploratory data analysis, feature engineering, time-series machine learning, feature-store management, model registration, recursive forecasting, API development, dashboard development, explainability, and CI/CD automation into a single end-to-end system.

The project uses approximately two years of hourly historical data and generates 70 engineered model features. Exploratory Data Analysis was performed before feature engineering to understand pollutant behavior, AQI persistence, weather relationships, temporal patterns, and data distributions. These findings guided the construction of lag, rolling, change, pollutant, weather, and temporal features.

Four machine learning approaches were trained during model development: Ridge Regression, Random Forest Regression, XGBoost Regression, and BiLSTM. The BiLSTM was investigated as a sequence-based deep learning approach for the temporal forecasting problem. However, the final production model-registration workflow contains the three selected regression models: Ridge Regression, Random Forest Regression, and XGBoost Regression.

Among the registered production candidates, XGBoost achieved the best documented test performance:

MAE = 1.3132

RMSE = 2.4022

$R^2 = 0.9943$

XGBoost was therefore selected as the champion model and registered as:
Pearls_AQI_XGBoost@champion

The production forecasting pipeline uses 96 hours of historical context and generates 72 recursive hourly predictions. These predictions are aggregated into three daily AQI averages for dashboard presentation.

Feast provides feature management and serving, PostgreSQL and Supabase provide cloud database infrastructure, MLflow and DagsHub provide experiment tracking and model management, FastAPI provides backend prediction services, Streamlit provides the user interface, SHAP provides explainability, and GitHub Actions automates the data and machine learning lifecycle.

PROJECT SUMMARY

Project: Pearls AQI Predictor

Purpose: AQI monitoring and 3-day forecasting

Location: Peshawar, Pakistan

Historical Data: Approximately 2 years

Raw Records: 17,544

Processed Records: 17,471

Model Features: 70

Prediction Target: Next-hour AQI

Models Trained: Ridge, Random Forest, XGBoost, BiLSTM

Models Registered: Ridge, Random Forest, XGBoost

Champion Model: XGBoost

Test MAE: 1.3132

Test RMSE: 2.4022

Test R²: 0.9943

Feature Store: Feast

Database: PostgreSQL / Supabase

Model Registry: MLflow / DagsHub

Forecast: Recursive 72 hours

Backend: FastAPI

Frontend: Streamlit

Explainability: SHAP

Automation: GitHub Actions

Data Source: Open-Meteo

Repository: Iqrajan2024/AQI_Predictor

Deployment: Streamlit Cloud + FastAPI Cloud